

Approaches to E2E Simulator Implementation: Lessons from CHIME Instrument Optical Module

April 15, 2026

Manfred Gawlas, Adrian Garcia, Antonio Gabriele, Ignacio Fernandez Nunez,
Jens Nieke

Document Approval

Role	Name	Signature	Date
Author	Manfred Gawlas		April 15, 2026
Reviewer			
Approver			

Change Record

Version	Date	Author	Description	Sections
Draft	22.12.2025	Manfred Gawlas	Structure and initial text.	All
Draft	15.01.2026	Manfred Gawlas	Full thing	All

Abstract

End-to-end (E2E) simulators play a central role in the development, verification, and validation of Earth observation mission performance within ESA. While their importance is universally recognized, the detailed implementation processes of E2E simulation software are often less formalized, presenting opportunities for further architectural optimization. ESA provides comprehensive high-level guidance through Reference Architecture, Interface, and Generic Requirements documents, alongside frameworks like Biblos3. However, their focus on high-level definitions often leads to mission-specific implementations, limiting reuse and increasing testing costs. This paper provides a concise architectural review of current E2E simulation practices in the ESA Earth observation context, focusing on implementation structures, development workflows, and operational limitations. Key functional and non-functional requirements are identified, together with common pain points that affect cost, reproducibility, and testability. A comparison is drawn with alternative E2E simulation practices employed in large-scale

scientific software systems, such as the ATLAS Athena framework at CERN. As a major contribution of this work, a prototype for the CHIME Instrument Optical Module (IOM) is presented, establishing a proof of concept for functional, Python-centric architectures. To address systemic challenges agency-wide, we propose shifting away from isolated implementations toward a standardized, two-pillar library architecture: an *Algorithmic Library* for abstracting mathematical operations, and a *Framework Library* for managing E2E-specific infrastructure and pipeline orchestration. These pragmatic recommendations aim to simplify simulator development, leverage modern hardware, and reduce long-term maintenance and verification efforts.

1 Introduction

End-to-end (E2E) mission performance simulators (E2ES) are indispensable tools in the development of modern Earth observation (EO) missions at the European Space Agency (ESA) [1, 2, 3, 4, 5], and are increasingly being adopted for projects at other agencies, including DLR and JAXA [6, 7]. These simulators are critical for assessing mission performance and verifying ground processing chains [2, 3, 4]. To ensure interoperability, ESA has successfully standardized the high-level architecture of these systems, particularly regarding module interfaces and the overarching framework provided by openSF [8, 9, 5, 10].

However, while external interfaces are well-defined by ESA, the internal implementation of these modules remains highly flexible, with lack of formalized guidance. Aspects such as coding style, library selection, and internal software architecture are often left to the discretion of individual contractors, although some specific recommendations exist, such as the use of OpenMP and OpenCL [5, 11]. This lack of standardization frequently leads to monolithic codebases where standard operations—such as convolutions, downsampling, or binning—are implemented directly within the mission-specific logic. Consequently, extracting and reusing code for subsequent missions can become challenging, as these standard functions become tightly coupled with the project-specific architecture.

Comment

I see that you mention this(Biblos3) in the first paragprah of the introduction. I would include a few lines on that here too To acknowledge existent status of documentation, we can add that there exist a Reference Architecture, Interface and Generic requiements documents, but they deal with high level definitions and does not discuss implementation details.

Comment

I do not fully grasp what you are after here.

You are making the point that, because of lack of abstraction, it is difficult to re-use code written for other missions?

Answer

Yes, but not only. Because some functions are not abstracted, their implementation bloats the code-base and requires testing at the code-base level. For example image you would want to write simple program that adds two arrays in python. You would likely just use numpy and use that it implements element wise $+$ operator on arrays, you would not want to implement it yourself, as it could be error-prone and require testing as well as you would have to provide this new huge function to the script, which would make the program bigger - and by extension harder to maintain. Even if you would need to implement those functions as they dont exists, you would likely want to have it in different part part of code or put in some simple library which you could just import.

Rather than re-implementing these standard functions for every new project, shifting toward a validated, generic E2E operations library (whether industry-owned, open-source, or ESA-maintained) could offer architectural advantages. As illustrated in Figure 1, decoupling these operations into an abstract library—similar to how one might call a standard ‘numpy’ function—provides several concrete benefits:

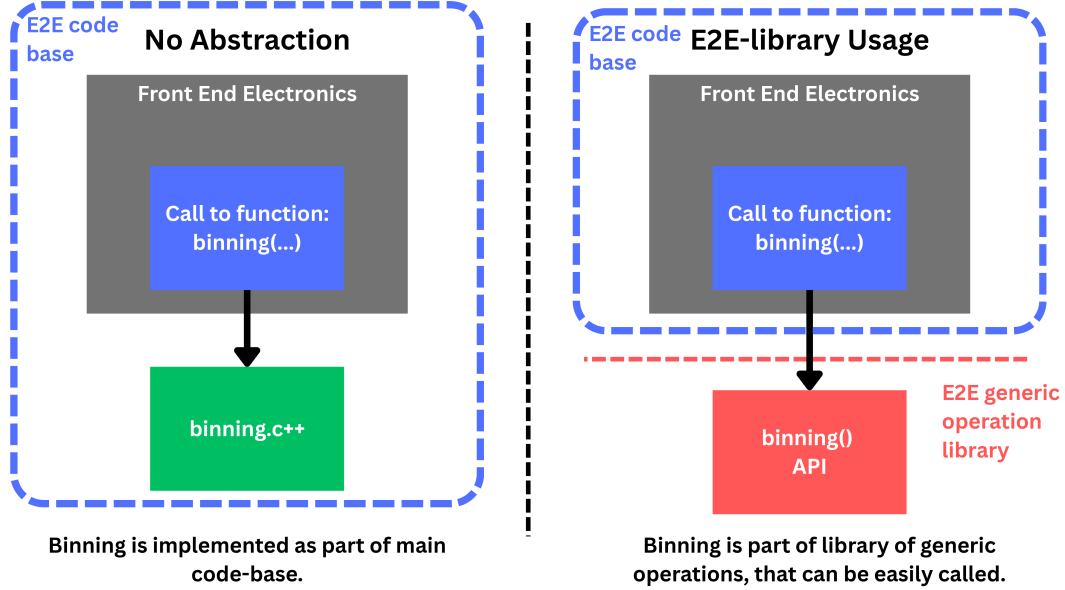


Figure 1: Comparison of implementation strategies. The traditional "No Abstraction" approach (left) embeds standard operations directly into the mission-specific codebase, requiring repeated testing and adaptation. The proposed "E2E-library Usage" approach (right) delegates these operations to a validated, external API, promoting reusability and keeping the main codebase lean.

These architectural challenges have practical consequences beyond software maintenance. The high complexity and limited transparency in the code create significant challenges for Instrument and System Performance engineers. These specialists, who often have backgrounds in physics and engineering rather than software engineering, are responsible for validating the underlying scientific algorithms [2]. When implementation details are obscured by overly complex software architecture, the gap between the physical model and its digital twin widens, increasing the risk of errors and driving up development costs.

The goal of this paper is to review current implementation and technological practices in the ESA EO context, identify core requirements and limitations, and propose simplifications aimed at reducing development and testing effort. Building upon existing ESA guidelines and the Biblos framework [11, 5, 12], this work incorporates insights from a recent ESA white paper and internal discussions. We compare current practices with alternative large-scale scientific frameworks, such as ATLAS Athena at CERN, to identify transferable lessons.

Finally, we propose a Python-centric architectural direction designed to produce cleaner, more readable code that helps to serve as its own documentation and supports multi-hardware options, ultimately facilitating easier validation for ESA staff and more efficient delivery for industrial contractors.

2 Background and Scope

To establish a common baseline, it is necessary to define the functional boundaries of the systems under discussion. According to the ESA E2E White Paper [2], an End-to-End Mission Performance Simulator (E2ES) is defined as:

"...chains of software tools developed to simulate the generation of the geophysical observables target scene, the whole observation process from space including orbit, pointing and guidance, the instruments, the on-board data generation and the processing of the outputs of these instruments to retrieve the estimate of the original observables."

In this context, the E2ES serves as a "digital twin" of the entire measurement chain. While the high-level orchestration of these chains is often handled by frameworks like openSF, this paper focuses specifically on the **module-level architecture**—the internal design and implementation of the individual building blocks (e.g., Instrument Simulation Modules) that constitute the chain.

2.1 Scope of the Discussion

The scope of this work is centered on the software engineering and operational aspects of E2ES development rather than the underlying physics. Specifically, we address:

- **Software Architecture and Technology:** Reviewing the current use of object-oriented module architecture, languages, and libraries, with a focus on moving toward a Python-centric stack that utilizes modern scientific computing tools developed in performant languages
- **Hardware Utilization:** Discussing the trade-offs between CPU-based processing and GPU-accelerated workflows to meet increasing performance demands.
- **Validation and Testing Workflows:** Proposing methods for automated error reporting and result analysis to replace labor-intensive manual verification and to properly analyze numerical errors.

- Documentation Practices: Exploring the transition from static, text-heavy implementation documentation toward self-documenting code and markdown documentation as part of the code repository.

Conversely, this paper does not cover mission-specific algorithm derivations or detailed physical modeling for particular Earth observation sensors.

Finally, the paper references a practical implementation case: the Instrument Optical Module for the CHIME (Copernicus Hyperspectral Imaging Mission for the Environment) E2ES. While this module did not adhere to all the proposed guidelines from its inception, the lessons learned during its development served as the catalyst for the requirements and architectural directions proposed in the following sections.

2.2 Target Audience

The recommendations provided here are intended for software implementation contractors, scientific algorithm developers, instrument performance engineers, simulator and processor engineers, and ground segment system engineers. The objective is to suggest modern tools and workflows for industry partners, while outlining expected technical standards for ESA staff. By lowering the implementation complexity, we aim to provide ESA staff with more transparent tools for verification while reducing the development burden on industrial partners.

3 Current E2E Module Architecture and Implementation

3.1 High-level overview of architecture

The highest-level architecture of E2ES within ESA can be described primarily by the functional role of its constituent modules:

- Instrument Performance Module - responsible for satellite and instrument behaviour.
- Ground Processor - L1 processing to L2 retrieval.
- Performance Assessment Module - responsible for comparing the input to the retrieved image in order to assess mission performance requirements.

This high-level structure is further decomposed into several modules, an example of this dataflow is illustrated in Figure 2.

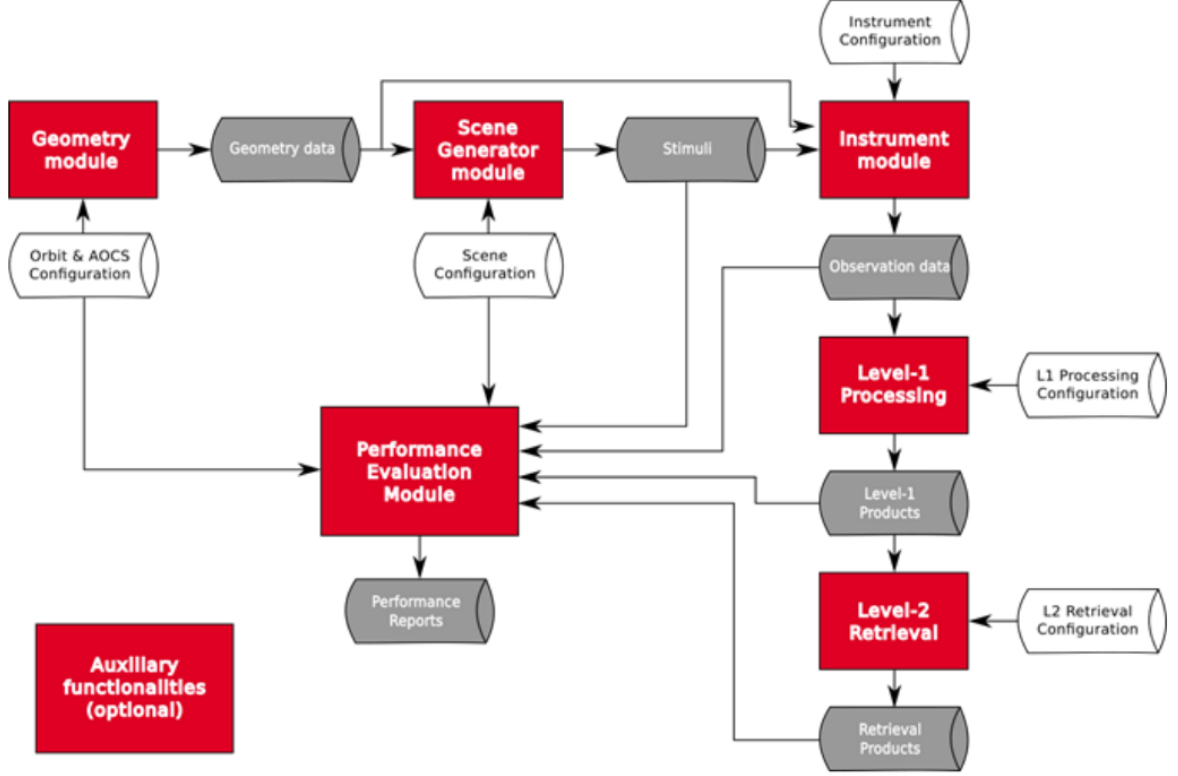


Figure 2: Generic dataflow at highest level of architecture [10].

While generic implementations of these modules exist for academia thanks to the Biblos3 project [5], their specific implementations must always be tailored to meet mission-specific requirements. It is at this implementation level that this paper focuses its attention.

3.2 Common Development Phases

The development of E2ES software follows an evolutionary approach that spans the entire mission lifecycle, progressing from simplified feasibility models to detailed, mission-specific implementations [2]. This progression can be characterized by several distinct stages:

1. **Initial Generic Implementation (Phase 0/A/B1):** A generic simulator is developed to assess mission feasibility and L2 retrieval concepts. This early version typically employs simplified error models applied to Scene Generation outputs [2].
2. **Mission-Specific Development (Phase B2/C/D/E1):** Detailed, mission-specific implementations are developed. While these may occasionally build upon the early

generic foundation, they are frequently developed independently under separate contracts, often by entirely different industrial consortia. Separate ATBDs are developed for individual modules (ISM, L1 processing, L2 retrieval), and the software is then implemented according to these algorithmic specifications [2].

Old

3. **Mission-Specific Development (Phase B2/C/D/E1):** The generic foundation is expanded into detailed, mission-specific implementations. Separate ATBDs are developed for individual modules (ISM, L1 processing, L2 retrieval), often by different industrial consortia. Software is then implemented according to these algorithmic specifications [2].

Comment

"is expanded into detailed"

It is not always the case that the phase 0/A/B1 simulator is expanded and reused in this phase.

This is usually managed as separate contracts and even the consortium developing the SW is different

Answer

I removed the word "expaned" and clarified as you suggested that this often includes independed separate contracts.

4. **Validation and Test Data Generation:** Throughout development, Test Data Sets (TDS) are created for systematic verification and validation of both individual modules and the integrated chain [2].
5. **On-Ground Calibration Support:** The E2ES supports On-Ground Calibration Activities, where measured data is used for characterizing instrument behavior and preparing final IOM module [2].
6. **In-Orbit Calibration and Verification:** During the commissioning phase, the GPP component of the E2ES is utilized to support In-Orbit Calibration and Verification (IOCV) activities, ensuring the system operates correctly post-launch.

Comment

We can add a bullet between 4 and 5. In phase E1 the GPP part of the E2E can be used for IOCV (In Orbit Calibration and Verification activities) during commisionig,

Answer

Added.

7. **Operational Ground Processing (Phase E2):** The toolchain transitions to supporting Ground Processor maintenance, algorithm evolution, and anomaly investigation during long-term operations [2].

Comment

I would be more open in the sentence. "If not done properly this phased approach may easily end in having a software architecture that leaves concepts as maintainance and testability in a second place "

If not managed properly, this phased approach may easily result in a software architecture that leaves concepts such as maintainability and testability as secondary priorities. As the codebase evolves across different mission phases and industrial teams, the natural accumulation of project-specific adaptations could sometimes create tension between rapid implementation and long-term development, directly motivating the architectural concepts proposed in this paper.

Old

This phased approach underscores the importance of maintainable, testable software architectures. The need for rapid algorithm iteration during development and subsequent long-term operational use creates tension between flexibility and performance—a challenge that motivates the architectural considerations discussed in later sections.

Comment

Why the phased approach underscores this?
Is your hipotesis that the software is being built with layer over layer "patching" the behaviour, without considering some rationalizatoin of functionalities or needed architectural changes to accomodate new capacities?

Answer

Yes, this is mainly my hipotesis, but I would not focus on the new architectural changes that are necessary to accomodate new capabilities, but on more down to earth symptoms. There are two main problems we both saw that underscored this in IOM.

First was the application of spatial jitter in telescope LSF, which when we inspected code we saw was supported across the whole preparation process and we assumed could be one of reasons there was misscalculation of PyISM TELE LSF. All of this was done based on some one if statement that was up in the code which we didn't see, which was taken first in constructor from the local configs "ENABLE JITTER PSF" option, which was enabled, but in reality futher down the line was later hardcoded to change the value to FALSE. This is explained in comment then, that this was not necessary option as something changed during one of the phases. Now I belive that this is one of examples why the phased approach underscores the importance of architecture, because if the preparation process would be simpler, and the LSF preparation would be generic, we would never need to look into it's details, we would look at few top level lines which call the generic version of preparation function and get the values needed(getting the flag would happen right above the call and hardcoded false would also happen line above it, not somewhere in constructor). In this way, it was also harder to test the code, as if you would look at constructor, you would assume that TELE LSF should be convolved with jitter PSF if you enabled that in local config.

The second example was getting access to new sampling for the operation of convolution-downsampling from the ACT SPE in SGM geometry to oversampled but in detector grid geometry. This was hardly maintainable, as to get access to the algorithm generating grid, we had to go through 4 or 5 different files, which changed that in between as well as everything was a reference. Now simpler architectural aproach is to have this new geometry defined by calling function in top level pipeline file, and then down the line have it fed to this resampling algorithm, with all changes in between forbidden and the defintions existing in same one part of pipeline.

I think both likely came from phased approach, where needed changes were stacked on top of each other, which led to new changes happening at different parts of pipeline.

Furthermore, it is critical to recognize the computational nature of E2ES software. In most cases, these are image processing applications that handle very large datasets. Given that processing images of this scale is both memory-intensive and computationally demanding, these modules are typically implemented in highly performant C/C++ [5, 10, 11], with parallelization support provided through OpenMP and OpenCL [11, 5].

3.3 Requirements

Functional and non-functional requirements for E2ES software are specified by ESA in [11]. Key requirements include:

- E2E-ARC-GEN-080 - Code shall be parallelized
- E2E-ENV-GEN-060 - Implementation in C or C++14 or newer

...it is explicitly required that in Phase B2/C/D the SW is implemented in a performant compiled language.

- E2E-ENV-GEN-090 - Ground processing parallelization through the use of OpenMP
- E2E-ENV-GEN-110 - GPU support through common APIs (e.g., OpenCL, CUDA)

3.4 Recurring Limitations and Pain Points

While the requirements outlined in Section 3.3 are understandable from a performance perspective, they often impose limitations that are not always aligned with practical development realities. Performance is not the only consideration—development cost, ease of maintenance, and accessibility to domain experts are equally important. This clash of domains is particularly evident in environments where ATBD preparation, software development, and validation are distributed across different locations and split between ESA and industrial partners [2].

Several recurring challenges have been identified through recent mission developments and internal discussions, notably within the CHIME and LSTM teams, who have advocated for more flexible technological approaches, especially in the context of Instrument Simulator:

1. **Language and Ecosystem Constraints:** Python is currently the most popular programming language, with extensive support for scientific computing and a rich ecosystem of data science tools [13, 14]. While ground processing chains may require peak performance, achieving comparable performance for Instrument Simulation Modules (ISM) or Scene Generation Modules (SGM) using Python with modern libraries has the potential to significantly reduce development costs. This is especially relevant given that many system experts routinely use Python but lack deep C++ expertise.
2. **GPU Acceleration:** Although requirement E2E-ENV-GEN-110 mentions GPU support, there are cases where the adopted architecture makes no use of the potential presented by GPU processing, leaning towards more conventional architectures that rely heavily on data transfer, thereby favoring CPUs. While industry conclusions in early project phases (e.g., five years ago) may have validly dismissed GPUs due to the

technological landscape and integration complexities of the time, the rapid advancement of GPU hardware and scientific computing ecosystems makes revisiting these trade-offs a priority today. A stronger challenge must be presented in the early phases of new projects to ensure this potential for orders-of-magnitude performance improvement is not lost from the start.

This potential is highly evident when benchmarking core simulator operations. To demonstrate this, preliminary experimental tests were conducted, evaluating fundamental E2E operations such as convolutions, gain application, and electronic offset corrections. As detailed in Appendix A, these tests demonstrate an average processing time reduction of X orders of magnitude and an X -fold improvement in cost-to-performance ratio when comparing modern GPU processing (e.g., NVIDIA H100) to standard CPU processing (e.g., 16-core Intel Xeon). While GPU usage can be limited by PCIe memory bandwidth [15], most use cases can mitigate this by pipelining multiple operations entirely on the device, avoiding excessive host-device data transfer [16, 15]. Providing both CPU and GPU execution paths—even if the GPU path only covers partial, computation-heavy pipelines—would offer critical flexibility, particularly when intermediate breakpoint files are not required during nominal runs.

Old

3. **GPU Acceleration:** Although requirement E2E-ENV-GEN-110 mentions GPU support, real implementations often lack this capability despite the potential for orders-of-magnitude performance improvements. This was evident in early testing of a simple per-pixel uncertainty estimation pipeline for the LSTM and CHIME L1C products at ESA, where preliminary results showed processing time reductions of 3–4 orders of magnitude (not accounting for host-device data transfer) when comparing GPU processing (H100) to CPU processing (16-core Intel Xeon). While GPU usage can be limited by PCIe memory bandwidth [15], most use cases do not involve excessive data transfer - as many operations can be performed in one pipeline at GPU - and could benefit substantially from GPU-accelerated image processing [16, 15]. Providing both CPU and GPU execution paths, even if GPU would be only partial, would offer flexibility for different operational scenarios, particularly when breakpoint file generation is not required during nominal runs.

Comment

" real implementations often lack this capability despite the potential for orders-of-magnitude performance improvements" We have only analyzed two cases and it may be the case that other simulators are making use of GPUs. Both for CHIME and LSTM a study early on the project was done to analyze if it was needed and industry conclusion was that it was not as beneficial, but that conclusion might have been wrong. In this case, I would be less generic on the phrasing and say: " there are cases where the architecture adopted makes no use of the potential presented by GPU processing, leaning towards more conventional architectures that rely heavily in data-transfer, therefore favoring CPUs. A stronger challenge can be presented in earlier phases of the project to not lose from start this potentially orders-of-magnitude performance improvement."

Comment

" This was evident in early testing of a simple per-pixel uncertainty" I do not know if this is the best example, the uncertainty computation is not part of the GPP, its a separate tool that can be implemented using this GPU approach that you proposes.

Answer

I have updated the phrasing using your exact suggestion so that we don't unfairly generalize all E2E implementations. I also added a sentence acknowledging that while relying purely on CPUs might have been a completely valid choice by industry 5 years ago(which the previous version lacked), it needs to be challenged today. Regarding the uncertainty tests, I completely agree with you. I removed it entirely, but likely will mention it later in Appendix if you think adding it and other tests would be good idea. I think it would strengthen argument significantly, especially taking into account that at IOM for example most of processing time was simple convolution with small kernel at telescope module, but also straylight will likely be of high time consumption and mathematically is quite a known operation - just convolution. If you think we should add that, kinda in similar style as I had in pointing uncertainty, I would need to test that at servers of laboratory at my Uni I work with that I mentioned, so I would put them in acknowledgments, but like those operations are on very generic math operation so there should not be problem with me testing that outside ESA if it is just convolution, with maybe sizes resembling those of CHIME and LSTM. But it will take a bit of time for me to do that, I have at this point 1D gain implemented in CPU and CUDA, rest will be also needed to be done.

4. **Performance Monitoring:** Automated support for timeliness metrics and runtime

performance visualization could be included as part of the baseline.

Comment

Replace by: "Automated support for timeliness metrics and runtime performance visualization could be included as part of the baseline."

5. **Manual Validation and Error Analysis:** The absence of automated report generation for results analysis and numerical error propagation places an unnecessary burden on validation teams. Automated tooling, implemented at the scientific library level, for these tasks would reduce manual effort and improve reproducibility.

Comment

I am not sure what are you thinking of in this bullet.
You are thinking of the testing of the SW and how the execution, assessment and reporting is done?

Answer

It's hard for me to explain this, as what the scope could be understood as. I have as part of my framework-library something that allows for full python report generation for each point of study and will provide you with script that you can run and get PDF in nice form. So instead of need for you to manually take BKP and load for example telescope LSF and see what is happening, you have automated software interface to get access very quickly and work on it, as well as show it also in form of report. This would help with lets say "visual" assesment and help to get quick the data so you can maybe do operations yourself and see if you get same results etc. Also I have additional mode that can run the pipeline in Numerical Error Assessment mode, which provides the report.py as well later PDF with relative errors etc, to check if float32 for example is not introducing some overly high errors compared to float64.

It would be easiest for me if we could talk about this, ask I don't know what the current version of what I have should be called - like is it automated result analysis or what term, as well as ask you what you think could be improved. I had a bit simpler idea of this before(the python report file would be part of pipeline), but your comment before was actually really usefull and I think now it is more valuable.

6. **Hardware-Software Co-Design:** While the OpenSF framework proposes the use of a file-based interface between modules, there are cases where it can be convenient to deviate from it since significant performance improvements could be obtained by making use of fast memory interfaces, optimized I/O strategies, and detached save threads.

Similar to the need for stronger early focus on architectural decisions mentioned regarding GPUs, these hardware implications must be evaluated from the project's origin.

Old

7. **Hardware-Software Co-Design:** The hardware architecture of ground processing systems is as critical as the software implementation itself. Fast memory interfaces, optimized I/O strategies, and detached save threads could yield significant performance improvements but are rarely considered in current architectural discussions, as are not in line with OpenSF framework.

Comment

This could be linked with the bullet on GPUs, were I proposed to modify adding that a harder early pushback on architectural decisions could be made

This are considerations that are currently done in the Operational Processor architecture but in many cases left aside for the Prototype Processor.

I agree that OpenSF proposes a file-based interface that might not be the best option for all cases. So I would write this differently:

"While the OpenSF framework proposes the use of a file-based interface between modules, there are cases where it can be convenient to deviate from it since significant performance improvement could be obtained by making use of fast memory interfaces, optimized I/o strategies and detached save threads."

Answer

I changed it to exactly like you suggested and signaled what you suggested with early phase pushback(tried not to use that word).

8. **Complexity Barriers for Domain Experts:** High levels of implementation complexity at the ISM level can directly impede the productivity of Instrument Performance engineers and system experts, who are responsible for validating the underlying physics. A key architectural challenge early in projects is the widespread reliance on heavily nested Object-Oriented Programming (OOP) paradigms, which can sometimes abstract algorithmic logic behind complex state management. Shifting towards a functional programming approach for physically meaningful operations—utilizing pure functions with explicit inputs and outputs rather than mutating references—improves code readability for non-software experts. This approach enables experts to work more efficiently and significantly simplifies the direct translation of mathematical ATBD equations into software implementations, without sacrificing modeling accuracy.

Old

9. **Complexity Barriers for Domain Experts:** Overly complex implementations at the ISM level hinder the productivity of Instrument Performance engineers and system experts, who are responsible for validating the underlying physics. Simpler, more readable implementations, without sacrificing modeling accuracy, would enable these experts to work more efficiently.

Comment

You are missing a point that can be included in this bullet. OOP is often used as paradigm to define the SW architecture and this is another challenge that we have to make early in the project. You have shown that a functional approach can be beneficial to improve how this non SW experts understand the code, while it also helps while translating ATBDs equations into actual SW implementations.

Answer

I updated this to adress your comment, focused that this is for physically meaningful operations, not all of pipeline.

10. **Documentation-Implementation Gap:** Traditional documentation is prone to diverge from the actual implementation, introducing validation risks. Adopting self-documenting code practices—where clear structure and meaningful naming serve as the primary reference—mitigates this discrepancy. Building on ESA’s ongoing exploration of LLMs for management tasks, highly readable code would enable these models to automatically parse and cross-check the programmatic implementation directly against mathematical ATBD specifications.

Properly addressing each one of these challenges can help to reduce development time, increase code transparency and ease the validation effort. A shift towards pragmatic architectural choices that balance performance, usability, and long-term maintainability is strongly advocated in this paper.

4 Alternative E2E Approaches: Lessons from Athena

This section presents a high-level comparison with E2E simulation practices adopted in large-scale scientific software systems, with particular focus on the ATLAS Athena framework at CERN. Key aspects such as Python-based configuration, collaborative development models, specialized tooling, and documentation practices are discussed. Relevant lessons and transferable practices for ESA Earth observation simulators are highlighted.

4.1 Python-Based High-Level Configuration

CERN acknowledges the need for Python tooling for high-level data analysis [14, 17, 18]. In Athena, high-level configuration—equivalent to defining specific processing pipelines such as the IOM in ESA context—is performed at the Python level [17, 18]. This approach allows scientists and engineers to assemble and customize components without requiring C++ algorithm modifications. Critically, these components can be reused across different experiments and analyses, reducing duplication of effort. This configuration strategy directly addresses the accessibility and flexibility needs identified for ESA E2ES implementations.

4.2 Open Collaborative Development Model

A distinguishing feature of Athena is its development model: the framework is collaboratively developed by CERN staff and academic researchers worldwide, fostering extensive peer review. In contrast, due to necessary contractual frameworks, at ESA, the involvement of the scientific community and software developers is typically restricted to those within the specific contracted consortium

Addressing comment

"In contrast, ESA operates a more closed system, which significantly limits capacity for broader technical discussions and community input."

At ESA we have some contractual limitations that do not let us be open to the world in certain aspects. But the consortium that is involved in an E2E development includes Universities (scientific community) and SW development. However, besides ESA staff reviewers.

I would rephrase this sentence as: "In contrast, at ESA the involvement of the scientific community and SW developers is closed to those involved in the consortium"

Testing, development, and integration in Athena are conducted through GitLab in an open-source manner. While Git-centric approaches are already standard within individual ESA projects, ESA personnel primarily operate as reviewers rather than active coders. Therefore, the most beneficial advancement would be establishing a centralized, cross-ESA Git repository hosting all E2E simulators. This would enable ESA and future consortia to easily extract and reuse validated code, significantly improving knowledge transfer and reducing redundant efforts across missions.

Old

Testing, development, and integration in Athena are conducted through GitLab in an open-source manner. Opening ESA E2E developments to the scientific community could potentially yield significantly decreased development costs through broader participation. At minimum, adopting a unified Git-centric workflow would enable ESA to monitor code development and progress in real time, improving oversight and facilitating knowledge transfer across missions and contractors, crucially decessing the path to edit, in case error is found at ESA side or something needs change, it would be much simpler to just send in a Pull Request and create an issue, rather than mean of email communication.

Addressing comment

This is for the whole At minimum paragraph.
Now we have a git-centric approach in the projects, but we are not in the role of coders. What would be interesting is to have a common gitrepo across ESA where all the E2E simulators are stored and from where we can take code for common developments.

Answer

I rewrote the paragraph to reflect the current state as you clarified, I removed the other suggestions and included your on the cross ESA accessible E2E projects.

4.3 Specialized Libraries and Tooling

CERN relies heavily on ROOT, a specialized C++ library for data management, and Gaudi, an event-processing framework. For ESA applications, developing an analogous, mission-agnostic algorithmic library could yield significant benefits. Built upon established tools like OpenCV or Dask, and functioning as a high-performance extension of NumPy, this library could provide:

- Built-in automated report generation
- Automated numerical error testing
- Integrated logger, timer and other utilities
- Clean, consistent interfaces for algorithm implementation

Supporting both direct C++ implementation and Python access via bindings (e.g., pybind11) would offer critical flexibility while maintaining a unified development experience.

Furthermore, this could leverage parallel computing libraries such as Dask, harnessing their computational power while abstracting the associated development complexity.

4.4 Documentation Practices

Athena maintains comprehensive, web-based documentation (accessible at <https://atlas-software.docs.cern.ch/>). Adopting a similar GitLab-based Markdown documentation system for ESA E2ES would be highly beneficial, particularly for scientific users and validation teams requiring rapid guidance on setup and workflows. This approach consolidates scattered information and ensures stakeholders always access the most current version. Furthermore, version-controlled Markdown makes correcting documentation errors significantly more efficient than drafting and distributing traditional PDF revisions.

5 Python-Centric Prototype: Lessons from CHIME IOM

As part of a recent internship with the CHIME team, a prototype Instrument Optical Module (IOM) was developed using a Python-based stack centered on Dask, Xarray, NumPy, and Numba [19, 20, 21, 22]. The objective was to provide a more flexible alternative to existing C++ implementations, particularly for supporting the configuration of the instrument Optical Chain with data coming from On-Ground Calibration Activities. To emphasize readability and maintainability, the architecture deliberately departed from traditional Object-Oriented Programming (OOP) paradigms in favor of a strict functional programming approach across all implementation levels. A generic form of this architecture is shown in Figure 4.

Addressing comment

"The architecture emphasized readability and maintainability through a functional programming approach across all implementation levels, and a generic form is shown in figure 4. "

We can highlight this aspect. The departure from OOP for a Functional Approach

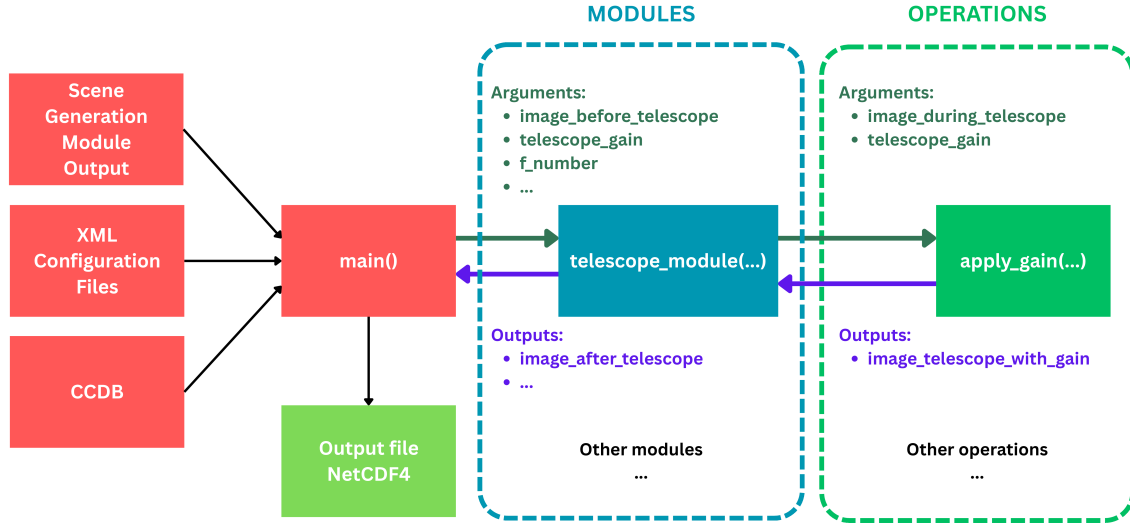


Figure 3: Simplified generic dataflow of the Instrument Module; modules are written primarily as pure functions, providing clear signatures for both the input and output.

Comment

maybe we can put a project agnostic example of a formula that can be part of an ATBD and how that formula is represented in a function, mapping the inputs and outputs to the signature of the function

I updated the diagram, I think simplest to understand is the telescope module, so I put it as more less project agnostic, even tho it is not completely.

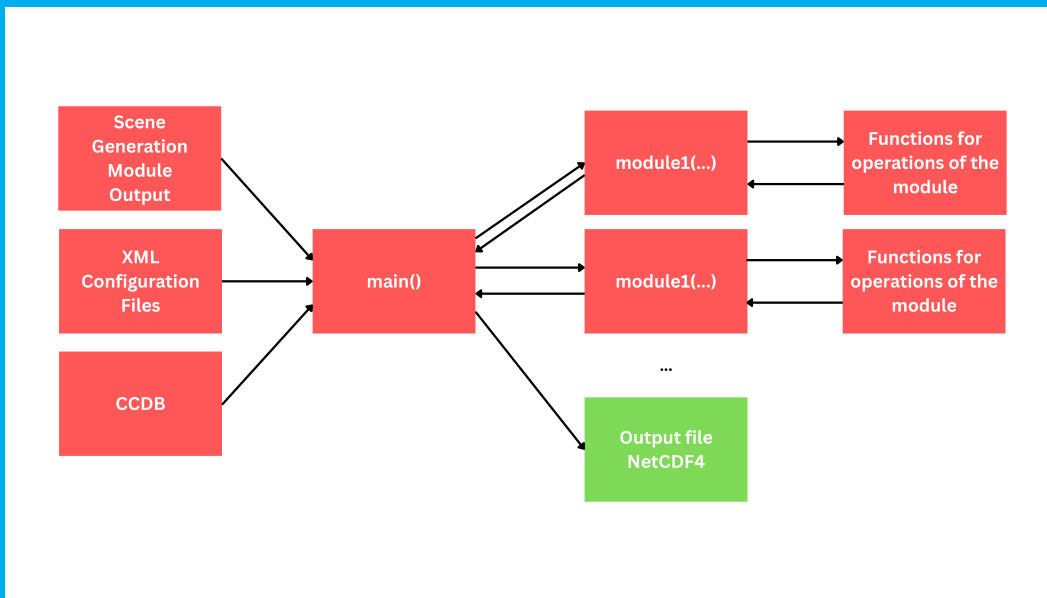


Figure 4: old figure

5.1 Observed Strengths

The prototype demonstrated several advantages:

1. **Memory Efficiency Through Chunking:** Dask’s native chunking support enabled significant reductions in RAM requirements [20]. Initial input images exceeding 50 GB—which could drive total memory consumption beyond 128 GB during processing—were successfully handled on virtual machines with only 32 GB of RAM. This capability would enable simulator deployment on standard computing infrastructure rather than requiring specialized high-memory systems.

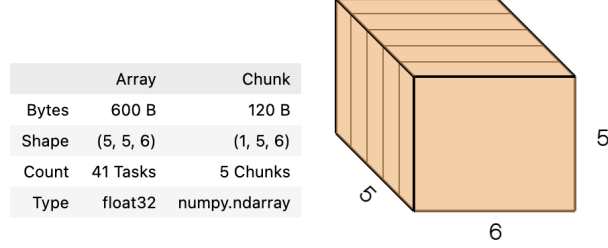


Figure 5: Simple example of Dask chunks; the image is processed as slices of a data cube, which are stored as standard NumPy arrays.

2. **Algorithm Clarity:** Implementation using NumPy and scientific Python libraries resulted in substantially shorter and more transparent code compared to equivalent C++ implementations.
3. **Simplified Data Management:** Dask’s lazy evaluation model streamlined checkpoint (BKP) file generation, consolidating multi-step save operations into single function calls.
4. **Comparable Performance:** Preliminary benchmarks on a representative test case showed processing times of 60–105 minutes for the Python implementation versus approximately 75 minutes for the nominal C++ version, suggesting performance parity for certain workloads. However, more comprehensive testing across diverse scenarios is required for validation.
5. **Extensibility:** Performance-critical functions can be selectively reimplemented in C++ or CUDA and integrated via Pybind11 [23], enabling targeted optimization while preserving Python’s high-level expressiveness. However, non-standard functions can also be implemented using Numba, which provides highly efficient JIT-compiled code for both CPU and GPU execution [22], while retaining Python’s simplicity. In cases where C++ implementation was utilized, pure Python fallback implementations were also provided, with the structure shown in Figure 6.

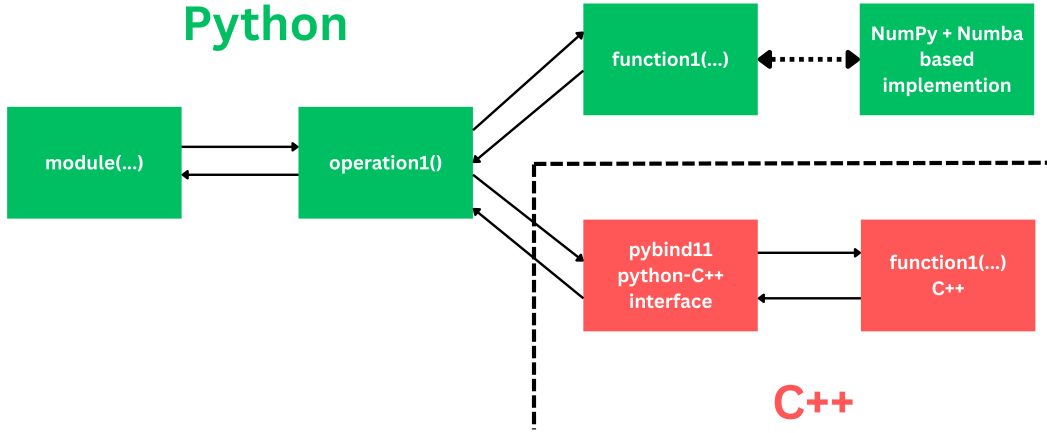


Figure 6: Function interfaces for the Python-C++ implementation.

Supporting On-Ground Calibration (OGC) activities is typically handled by integrating specific execution paths directly into the main processing pipeline. However, to avoid modifying the core codebase while detailed OGC requirements were still undefined, an alternative wrapper script approach was tested in the prototype.

As illustrated in Figure 7, this script utilized "monkey-patching" to dynamically alter code at runtime. By acting as an external runner, it modified how the simulator perceived the Characterization and Calibration Data Base (CCDB), overrode specific function signatures, and injected new parameters on the fly, providing extensive configuration freedom without altering the underlying `pysim` codebase.

While this method successfully isolated OGC modifications, the resulting dynamic complexity was significant. Upon reflection, the standard practice of embedding OGC-specific logic inside the main pipeline is generally simpler and more maintainable. Although this requires modifying the core code, the added internal complexity could likely be easily managed if the top-level architecture utilizes the explicit, functional approach discussed earlier.

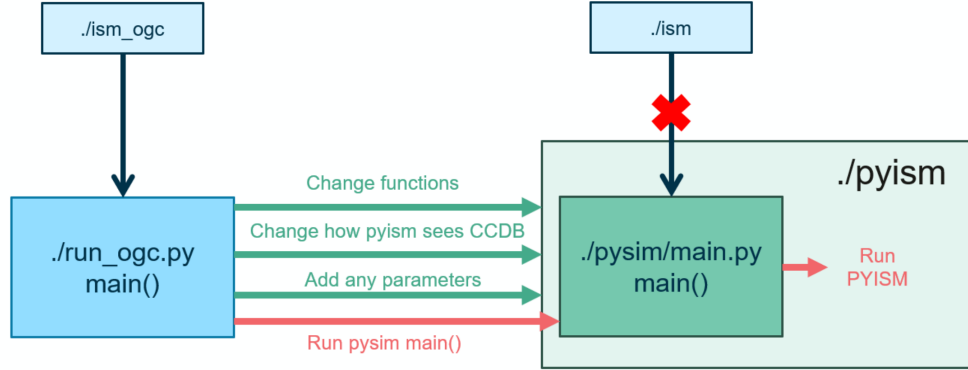


Figure 7: Alternative OGC support architecture using an external wrapper script. The `run_ogc.py` script dynamically patches functions, modifies CCDB visibility, and injects parameters before execution, avoiding direct modifications to the core `pysim` codebase.

Comment

"The OGC support was done as runner script for the monkey patching functions of the main IOM runner - equivalent of Athena configuration [17]."

For an external reader this is not clear.

Maybe we have to say somethign more generic, that is that the OGC support required that the signature of certain functions had to be modified because of the input parameters to be used. To overcome this a monkey-patching function approach was followed, which consisted in ... (to complete)

And add a diagram

5.2 Identified Limitations

Several challenges emerged during development:

1. **Error Diagnostics:** Dask generates verbose stack traces that can obscure the root cause of failures, complicating debugging efforts.
2. **Backend Instabilities:** The netCDF4 and HDF5 backends occasionally failed when parallel workers attempted concurrent writes, requiring workarounds with zarr.
3. **Overhead and Control:** Dask's multiprocessing layer, coupled with Dask graph generation, introduces significant data size-independent overhead and limits low-level execution control compared to native parallelization frameworks like OpenMP.

Overall, this experiment suggests that a dedicated, high-performance library, written in C++/CUDA and equipped with essential utilities, could significantly accelerate simulator development while providing the necessary infrastructure support.

A similar library, featuring a C++ backend API tailored specifically for the ESA E2E use case, could also be employed. In this context, chunking may not be strictly necessary, as images handled during ground processing are typically of significantly smaller size than those ingested by the IOM.

6 Discussions and Recommendations

The analyses in the preceding sections acknowledge that current E2E simulators are highly complex software systems that have been reliably delivered using established approaches. However, the Python-based implementations explored in this study present a viable alternative capable of addressing underlying systemic issues related to maintainability, transparency, and testability.

As a major contribution of this work, the CHIME IOM prototype establishes a proof of concept for Python-centric ISM E2ES architectures. While it successfully demonstrates the benefits of a functional, Python-driven approach, further work is needed to develop robust, production-ready frameworks. To address these systemic issues at an agency-wide scale and shift away from isolated, project-specific implementations, we propose the adoption of a standardized, two-pillar library architecture.

6.1 A Two-Pillar Library Architecture

Rather than building simulators based on generic modules and frameworks for each mission, future developments should rely on two distinct, complementary libraries to strictly separate mathematical implementation from pipeline orchestration.

6.1.1 The E2E Algorithmic Library

The first proposed pillar is a purely algorithmic library that abstracts standard Earth observation processing operations (e.g., convolutions, resampling, gain applications or ADC conversion). This library would provide standardized endpoints for both Python and C++, with seamless underlying support for CUDA to enable GPU acceleration for performance-critical operations.

This algorithmic library directly synergizes with the proposed cross-mission ESA Git-Lab repository discussed previously. Consortia and ESA engineers could extract well-tested monolithic code from past E2E simulators, write generic wrappers, and contribute them to this centralized library. As introduced in Figure 1, decoupling these operations into an abstract library provides concrete benefits:

- **Reduced codebase size:** Mission-specific repositories remain lean, focusing purely on high-level orchestration rather than low-level mathematical implementations.
- **Elimination of redundant testing:** Once a generic function is validated and approved on the ESA side, it can be safely reused across projects without requiring repeated unit testing at the project level.
- **Cross-mission reusability:** Industrial partners can easily port fundamental functionalities to new mission requirements, significantly reducing development time and costs.

The level at which these interfaces are proposed is demonstrated in Figure 8. The library would provide simplified interfaces for array handling without attempting to rewrite established tools like Xarray or Dask, while offering standardized testing interfaces to easily visualize data.

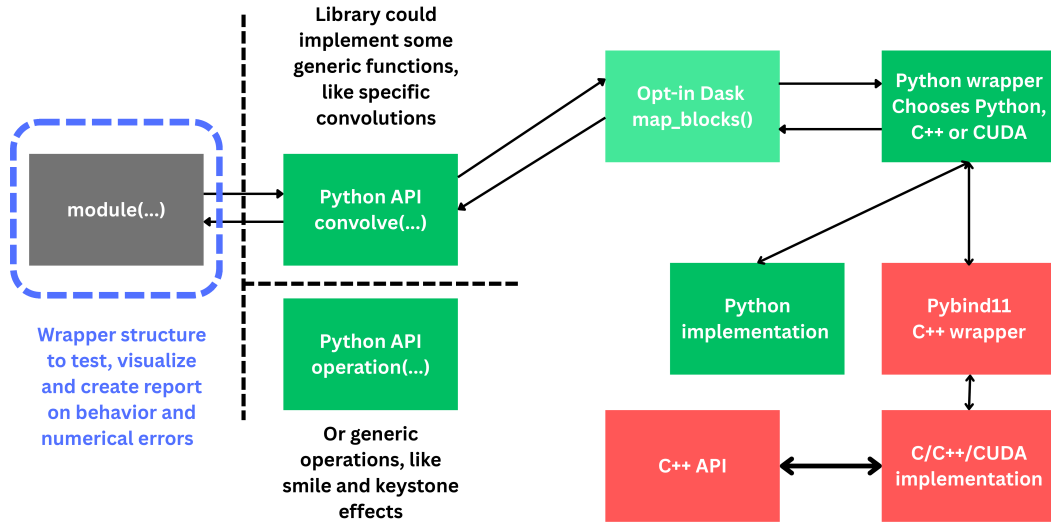


Figure 8: Example architecture of the proposed Python/C++ E2E Algorithmic Library. It provides simplified interfaces for common functions, automated testing, and visualization, abstracting the implementation complexity away.

6.1.2 The E2E Framework Library

The second pillar is a framework library dedicated strictly to handling E2E-specific software infrastructure. Instead of each consortium providing their own implementations of parsers

and I/O handlers, this framework would manage standard APIs, ingest required configuration files (e.g., OpenSF compliance), and provide an intuitive interface to assemble new pipelines using the abstracted components from the Algorithmic Library.

Crucially, this framework would natively support automated numerical error propagation throughout the entire processing chain. It would standardize simulator outputs by automatically tracking timeliness metrics, performing automated Quality Assurance for numerical stability, and generating customizable, python script reports, capable of generating automated PDFs, at each pipeline stage. This would directly address the limitations in transparency and performance monitoring identified in current operational implementations.

6.2 Summary of Recommendations

Based on the preceding analysis, the following specific actions are recommended for future E2E developments:

1. Adopt a Python-centric hybrid stack by utilizing the proposed two-pillar library architecture, using Python for configuration and pipeline assembly while invoking compiled C++/CUDA algorithms for heavy image processing.
2. Enforce hardware-software co-design from early project phases, explicitly evaluating GPU acceleration and optimized I/O strategies to prevent performance bottlenecks.
3. Standardize simulator telemetry by mandating automated timeliness tracking, numerical error propagation, and automated QA report generation natively through the framework library.
4. Establish a centralized, shared ESA Git repository to extract common algorithms, facilitate continuous review, and build the foundation for the cross-mission Algorithmic Library.
5. Leverage AI validation tools to verify consistency between the readable, functional codebase and the mathematical ATBD specifications.

7 Internal Physical Modeling Suggestions(FOR CHIME TEAM ONLY)

The complexity of the IOM implementation does not really come from the underlying physics, which are relatively straightforward, especially when ATBD is already provided. Aside from specific non-linear detector effects, ATBD primarily relies on four standard mathematical operations:

- 1D and 2D resampling (interpolation).
- 1D convolution over 2D arrays.
- 2D convolution over 1D arrays.
- Element-wise multiplication.

Despite mathematical simplicity of most operations, the current software implementation is often difficult to verify. This difficulty arises from the low-level management of grids and margins. Almost every operation requires re-calculating indices or handling margins manually, which leads to inconsistent implementations and potential numerical errors. Good example is usage of `arange` function for `float32`, which in our case generated the numerical error of 0.006% for final elements, compared to linear space (`np.linspace`). This error was only error seen at PyISM and OPSI functions corectly.

To address this, I would propose two architectural simplifications, tho it is likely not feasible to refactor the existing code now:

- **Standardized Grid Abstraction:**
The handling of array margins should be abstracted into a single, standardized function. Currently, different parts of the code calculate margins differently(telescope LSF has margins of 2, while HSI LSFs have only of 1). From a physical perspective, minor differences in margin inclusion should not significantly impact the result(as we dont know what happens there as it was not measured); if they do, it implies an error in how boundary is modeled. Standardizing this logic removes the risk of "off-by-one" errors and index mismatches.
- **Early Resampling Strategy:**
The current architecture performs spectral resampling from SGM to oversampled detector grid late in the chain (at the Detector module level), requiring more complex convolution logic to align data(this perhaps is one of the hardest parts of code to understand). I suggest moving this resampling step to the very beginning of the pipeline, immediately after the Scene Generation Module (SGM) in Motion module. By ensuring the input data matches the final target spectral grid from the start, we can eliminate the complex, error-prone code currently required to handle coordinate transformations later in the chain.

To conclude, the hard part of the simulator, is not the physical modeling(except Smile and keystone which are most copmlex operations at IOM level) when you have access to ATBD, but often the implementation of coordinate transformations. By abstracting grid management and optimizing the order of operations, the code could becomes more readable and less prone to the hidden indexing and numerical errors.

References

- [1] Fernando E. Aleman, Laura Zamora, Claudia Isola, Carolina Tenjo, Karl Segl, Stephane Guillas, Jose Luis Garcia, Jens Niecke, Antonio Gabriele, Adrian Garcia, Ignacio Fernandez Nunez, Miguel Morata, Jochem Verrelst, Adrian Jacinto, Cinzia Panigada, Massimo Musacchio, Stefano Pignatti, Alessandra Nguyen, Andrea Taramelli, Emiliana Valentini, and Giulia Tagliabue. CHIME’s end-to-end simulator (CHEES): simulating, testing and generating hyperspectral products. In Sachidananda R. Babu, Arnaud Heliere, and Tomoyuki Urabe, editors, *Sensors, Systems, and Next-Generation Satellites XXIX*, volume 13667 of *Proceedings of SPIE*, page 136670G. SPIE, 2025.
- [2] White paper on end-to-end mission performance simulation chains. White Paper PE-RP-ESA-GS-0754, European Space Agency, Noordwijk, The Netherlands, November 2025. ESA UNCLASSIFIED – For Official Use.
- [3] Luca Sgheri, Claudio Belotti, Maya Ben-Yami, Giovanni Bianchini, Bernardo Carnicero Dominguez, Ugo Cortesi, William Cossich, Samuele Del Bianco, Gianluca Di Natale, Tomas Guardabrazo, Dulce Lajas, Tiziano Maestri, Davide Magurno, Hilke Oetjen, Piera Raspollini, and Cristina Sgattoni. The forum end-to-end simulator project: architecture and results. *Atmospheric Measurement Techniques Discussions*, 2021. Preprint. Discussion started 2 September 2021. © Author(s) 2021. Licensed under CC BY 4.0.
- [4] Maria J. Sanjuan-Ferrer, Jalal Matar, Marc Rodriguez-Cassola, Pau Prats-Iraola, Matteo Nannini, Andre Barros Cardoso da Silva, Rolf Scheiber, Riccardo Piantanida, Andrea Recchia, Martin Steinisch, Davide Giudici, Luca Nardecchia, Marta Bucciarelli, Antonio Bauleo, and Silvia Mezzasoma. The rose-1 end-to-end performance simulator and ground processor prototype tools: An overview. In *Proceedings of the European Conference on Synthetic Aperture Radar (EUSAR)*. IEEE, 2024. Contact author: Maria J. Sanjuan-Ferrer (DLR).
- [5] Robert Kedzierawski, Wojciech Oryszczak, Karolina Szczepankiewicz, Adriano Camps, Hyuk Park, and Pedro Garcia. Biblos for simulation of the earth observation missions. page 62, 02 2020.
- [6] K. L. Segl et al. Eetes: An end-to-end image simulation tool applied to the enmap hyperspectral mission. In *Proceedings of the IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, pages 5025–5028. IEEE, 2012.
- [7] Takahiro Kawashima, Akira Iwasaki, Toshiyoshi Kimura, and Tadahito Mizutani. End-to-end performance simulator for greenhouse gas observation sensor. In *Sensors, Systems, and Next-Generation Satellites XXV*, volume 11858 of *Proceedings of SPIE*, page 118581A. SPIE, 2021. SPIE Remote Sensing 2021, Online Only.

- [8] DME and ESA OpenSF Team. Esa generic e2e simulator interface control document. Interface Control Document PE-ID-ESA-GS-464, European Space Agency, March 2023. Status: Final. Distribution: All.
- [9] Rui Mestre. Open simulation framework interface control document. Interface Control Document openSF-DMS-ICD-001, openSF, May 2018.
- [10] ARCHEO-E2E Teams. Eo e2e2 reference architecture. Reference Architecture GMV-ARCHEO-E2E-TN-002, European Space Agency, March 2018. Prepared by ARCHEO-E2E teams at GMV, Aresys, Universidad de Valencia, Universitat Politecnica de Catalunya, and ESA. Approved and authorized by Cristina de Negueruela Aleman. GMV/ESA 2018; all rights reserved.
- [11] EOP-PEP Team. Generic e2e performance simulator and l1/l2 processor: Requirement document and inputs to sow. Requirement Specification PE-TN-ESA-GS-402, European Space Agency, May 2020. Status: Approved/Applicable. Distribution: ESA UNCLASSIFIED – For Official Use.
- [12] GMV. Biblos3 executive summary report. Executive Summary Report GMV-BIBLOS3-D21_ESR, GMV, June 2023. Internal code: GMV 23202/23 V1/23. Approved by Patryk Jacek Wancerz, authorized by Robert Kędzierawski.
- [13] Wikipedia Contributors. Python (programming language). [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)), 2025. Accessed: 27 Dec 2025; states that Python was the most popular programming language as of 2022 and is widely used in data science and machine learning.
- [14] Choi, KyungEon, Feickert, Matthew, Hartmann, Nikolai, Heinrich, Lukas, Held, Alexander, Kourlitis, Evangelos, Krumnack, Nils, Stark, Giordon, Vigl, Matthias, and Watts, Gordon. Building a columnar analysis demonstrator for atlas physlite open data using the python ecosystem. *EPJ Web Conf.*, 337:01359, 2025.
- [15] Costin Vasile, Andrei-Alexandru Ulmămei, and Calin Bira. Image processing hardware acceleration—a review of operations involved and current hardware approaches. *Journal of Imaging*, 10:298, 11 2024.
- [16] Daniel Strigl, Klaus Kofler, and Stefan Podlipnig. Performance and scalability of gpu-based convolutional neural networks. pages 317–324, 02 2010.
- [17] ATLAS Collaboration. Configuration - athena documentation. <https://atlas-software.docs.cern.ch/athena/configuration/>. Accessed: 2026-01-13.

- [18] Johannes Elmsheuser. ATLAS ATHENA Configuration. Presentation at the ePIC Software Meeting, July 2023. With material from Tadej Novak (DESY), Walter Lampl (University of Arizona), Scott Snyder (BNL), Vakho Tsulaia (LBNL).
- [19] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with numpy. *Nature*, 585(7825):357–362, September 2020.
- [20] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. *SciPy 2015*, 2015.
- [21] Stephan Hoyer and Joe Hamman. xarray: N-d labeled arrays and datasets in python. *Journal of Open Research Software*, 5(1):10, 2017.
- [22] Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: a llvm-based python jit compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, LLVM '15, New York, NY, USA, 2015. Association for Computing Machinery.
- [23] Wenzel Jakob, Jason Rhinelander, and Dean Moldovan. pybind11 – seamless operability between c++11 and python. <https://github.com/pybind/pybind11>, 2017.